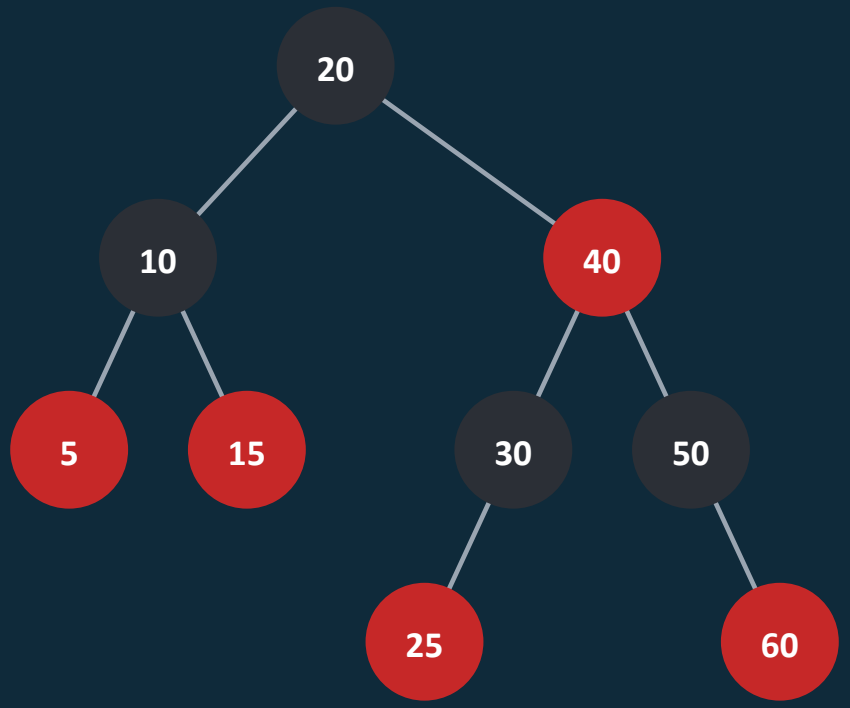
AMAZON ENGINEERING CHALLENGE · IMD0039

Indexação de encomendas na Black Friday

AVL x Rubro-Negra: implementação, testes e recomendação técnica



Leonardo Rosendo Faustino
Matrícula 20250027331
Estruturas de Dados Básicas II · UFRN/IMD · Outubro de 2026

Repositório: https://github.com/LeoRosendof/Sistema-de-Arvores

01

A proposta: indexar encomendas em tempo real

Na Black Friday, a logística registra, rastreia e dá baixa em centenas de milhares de encomendas. Cada encomenda tem um ID único, gerado em sequência, e o índice precisa responder rápido em qualquer momento do pico.

Objetivos do protótipo

- Registrar, localizar e remover IDs em O(log n), qualquer que seja a ordem de chegada
- Implementar Árvore AVL e Árvore Rubro-Negra do zero, com as mesmas operações
- Testar as duas com os mesmos dados e as mesmas sequências de operações
- Recomendar uma estrutura com base em evidências medidas

O(log n)

custo garantido para inserir, buscar e remover, mesmo no pior caso

IDs em sequência

a ordem natural de chegada é o pior caso de uma árvore de busca comum

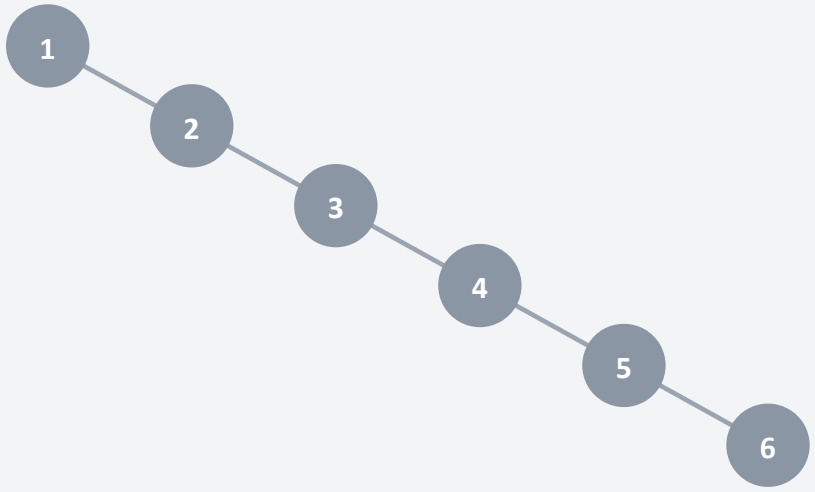
2 estruturas

AVL e Rubro-Negra atrás da mesma interface, com os mesmos testes

02

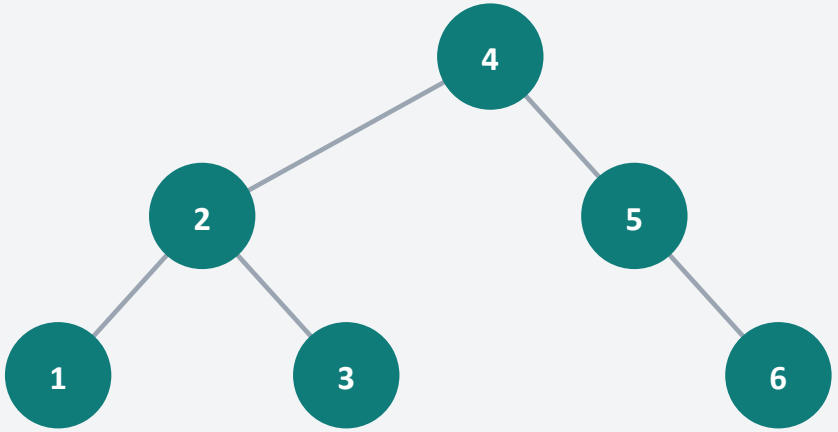
O desafio: a ordem de chegada importa

ABB comum, IDs 1 a 6 em ordem



Altura 6: virou uma lista encadeada

Árvore balanceada, mesmos IDs



Altura 3: rotações mantêm a árvore baixa

Com 10.000 IDs sequenciais (medido no protótipo):

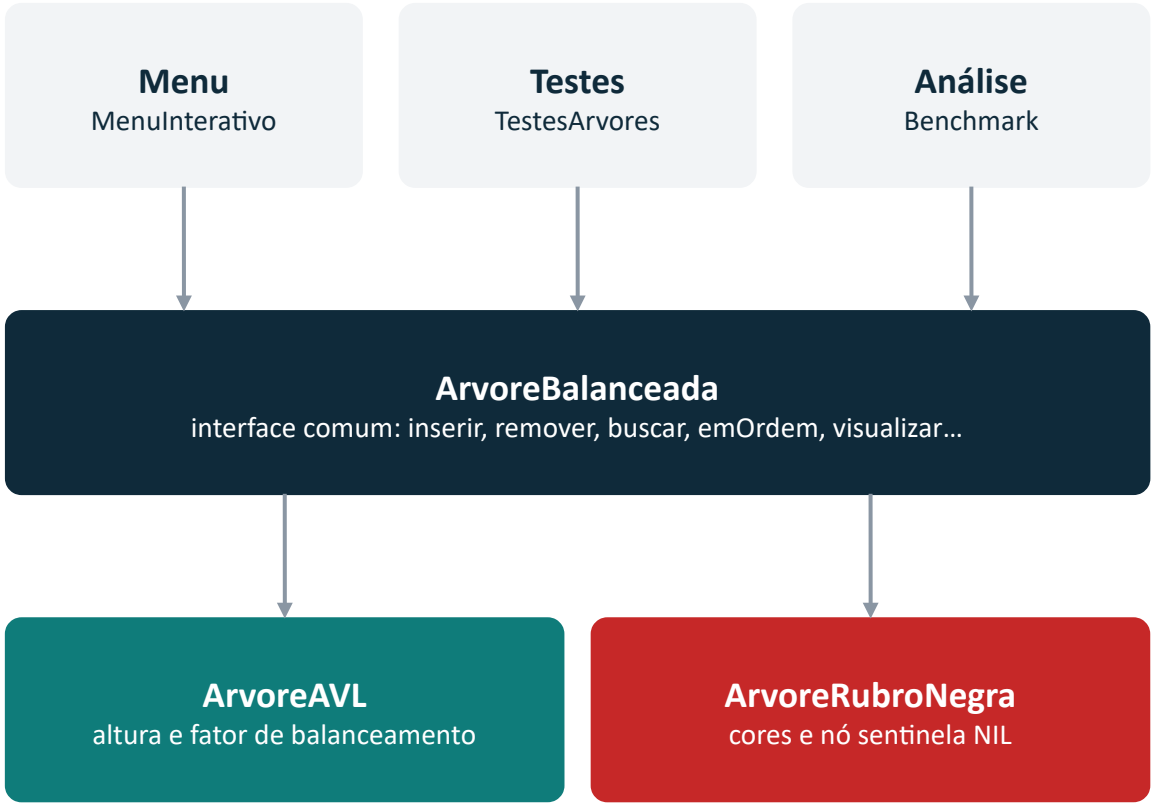
10.000
altura da ABB comum: busca O(n)

14
altura da AVL

24
altura da Rubro-Negra

03

A solução: um protótipo, duas estruturas



Mesma interface: o menu, os testes e a análise executam exatamente as mesmas operações nas duas árvores.

Funcionalidades

- Inserção e remoção de um ou vários IDs
- Busca, com nº de comparações, filhos e cor
- Percurso em ordem (IDs crescentes)
- Visualização: FB na AVL, cores na Rubro-Negra
- Modo detalhado: rotações e cores passo a passo
- Estatísticas e verificação das propriedades
- Comparação AVL x Rubro-Negra lado a lado

```
java -jar dist/arvores-balanceadas.jar
  [--avl | --rn | --testes | --benchmark]
```

03 Decisões de implementação

Situação	Decisão adotada
Inserir ID repetido	Rejeitado com aviso; a árvore não muda (ID de encomenda é único)
Remover ou buscar ID inexistente	Informa "não existe"; nenhuma alteração na estrutura
Remover nó com dois filhos	Substituído pelo sucessor em ordem (menor da subárvore direita)
Árvore vazia	Nenhuma operação falha; busca e remoção devolvem "não encontrado"
Entrada inválida no menu	Valor ignorado com aviso; o programa continua
AVL	Recursiva; cada nó guarda a altura; rebalanceia na volta da recursão
Rubro-Negra	Iterativa (Cormen, cap. 13), com ponteiro para o pai e sentinela NIL preto
Medida de altura	Em níveis: vazia = 0, um nó = 1
Contagem de rotações	Rotação dupla = 2 rotações simples, nas duas árvores
Recolorações	Contadas só quando o nó realmente muda de cor

04

Árvore AVL: balanceamento por altura

FB = altura(esq) − altura(dir)

Invariante: FB ∈ {−1, 0, +1} em todos os nós

- Cada nó guarda a própria altura
- Na volta da recursão, as alturas do caminho são atualizadas
- O primeiro nó com |FB| = 2 é corrigido com uma rotação simples ou dupla
- Inserção: no máximo 1 correção. Remoção: até 1 por nível, O(log n)
- Altura garantida: $h < 1{,}44 \cdot \log_2(n + 2)$

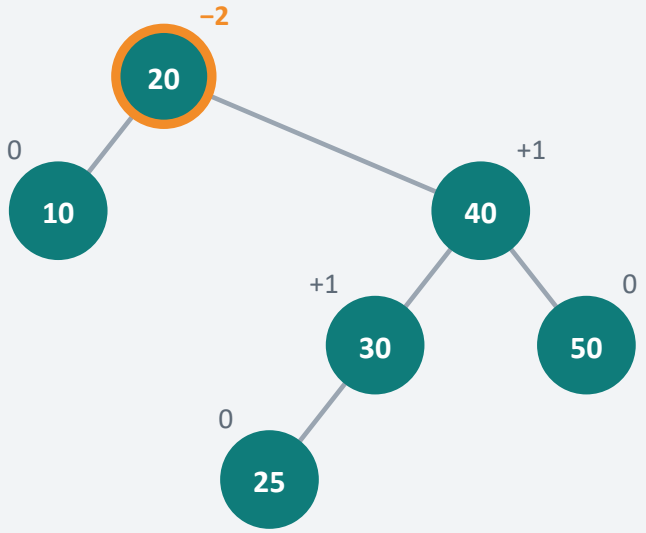
Caso	Condição	Correção
Esquerda-Esquerda	FB = +2 e FB(esq) ≥ 0	rotação simples à direita
Direita-Direita	FB = −2 e FB(dir) ≤ 0	rotação simples à esquerda
Esquerda-Direita	FB = +2 e FB(esq) < 0	esquerda no filho, depois direita no nó
Direita-Esquerda	FB = −2 e FB(dir) > 0	direita no filho, depois esquerda no nó

Os dois últimos casos são rotações duplas: o filho pende para o lado oposto, então uma rotação simples apenas espelharia o desequilíbrio.

04

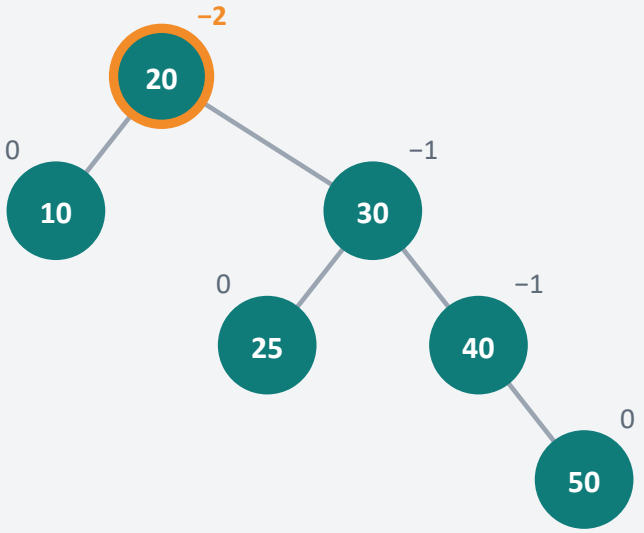
AVL: rotação dupla ao inserir o ID 25

1. Após inserir 25



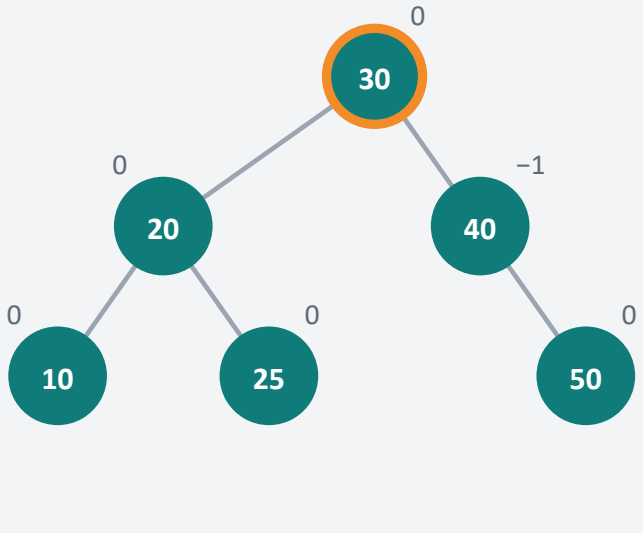
Nó 20 fica com FB = −2 e o filho 40 pende para a esquerda (+1): caso Direita-Esquerda

2. Rotação à direita em 40



30 sobe no lugar de 40; agora o desequilíbrio está todo à direita

3. Rotação à esquerda em 20



30 vira a raiz, nenhum nó fica com FB = ±2 e a altura cai de 4 para 3

Sequência executada no protótipo: 10, 20, 30, 40, 50, 25. Números ao lado dos nós = fator de balanceamento; contorno laranja = nó em correção.

05

Árvore Rubro-Negra: balanceamento por cores

1. Todo nó é vermelho ou preto
2. A raiz é preta
3. Toda folha NIL é preta
4. Nó vermelho não tem filho vermelho
5. Todo caminho até as folhas tem o mesmo número de nós pretos (altura negra)

Consequência: $h \leq 2 \cdot \log_2(n + 1)$. O caminho mais longo tem no máximo o dobro do mais curto.

Inserção (nó novo entra vermelho)

Caso 1: tio vermelho
recolore pai, tio e avô; o problema sobe

Caso 2: triângulo
rotação no pai, que vira o caso 3

Caso 3: linha
recolore e rotaciona no avô; termina

≤ 2 rotações

Remoção (saiu um nó preto)

Caso 1: irmão vermelho
recolore e rotaciona no pai

Caso 2: sobrinhos pretos
irmão fica vermelho; sobe

Caso 3: sobrinho próximo vermelho
rotação no irmão, vira o caso 4

Caso 4: sobrinho distante vermelho
recolore e rotaciona; termina

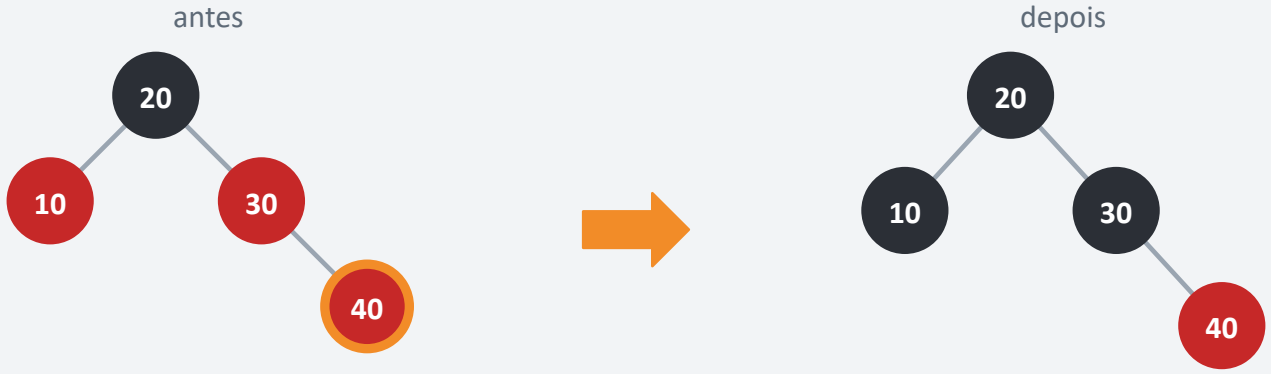
≤ 3 rotações

05

Rubro-Negra: recoloração e rotação

Inserir 40 · caso 1

40 e o pai 30 são vermelhos. O tio 10 também é vermelho: pai e tio ficam pretos, o avô fica vermelho e, por ser a raiz, volta a preto. Nenhuma rotação.



Inserir 50 · caso 3

50 e o pai 40 são vermelhos, o tio (NIL) é preto e 30-40-50 formam uma linha: 40 fica preto, 30 fica vermelho e há rotação à esquerda em 30.



06 Demonstração: Árvore AVL (modo detalhado)

```
java -jar arvores-balanceadas.jar --avl

Opção: 1
ID(s) a inserir: 10 20 30 40 50 25
  …
ID 30 inserido.
  > Nó 10 com FB=-2: caso Direita-Direita (rotação simples à esquerda)
  >   rotação à esquerda em 10: 20 sobe e vira raiz da subárvore
  …
ID 25 inserido.
  > Nó 20 com FB=-2 e filho direito 40 com FB=+1: caso Direita-Esquerda (rotação dupla)
Opção: 1
ID(s) a inserir: 30
ID 30 já existe: inserção ignorada (IDs de encomenda são únicos).
Opção: 3
ID(s) a buscar: 25 99
ID 25 encontrado (filhos: 0) após 3 comparação(ões).
ID 99 não encontrado (3 comparação(ões)).
Opção: 2
ID(s) a remover: 99      ← inexistente
ID 99 não existe: nada foi removido.
ID(s) a remover: 10      ← folha
ID(s) a remover: 40      ← um filho
ID(s) a remover: 30      ← dois filhos
  > Nó 30 tem dois filhos: será substituído pelo sucessor 50
  > Nó 50 com FB=+2 e filho esquerdo 20 com FB=-1: caso Esquerda-Direita (rotação dupla)
```

visualização após as inserções

```
        /-- 50 (fb=0)
    /-- 40 (fb=-1)
    30 (fb=0)
    |   /-- 25 (fb=0)
    \-- 20 (fb=0)
        \-- 10 (fb=0)

Percurso em ordem (6 IDs):
  10, 20, 25, 30, 40, 50
```

estatísticas ao final

```
Opção: 6
Altura ..............  2
Rotações ............  6
  (esq.: 4, dir.: 2)
Propriedades .........  OK
```

06 Demonstração: Rubro-Negra (mesma sequência)

```
java -jar arvores-balanceadas.jar --rn

Opção: 1
ID(s) a inserir: 10 20 30 40 50 25
  …
ID 40 inserido.
  > Inserção caso 1 (tio 10 vermelho): pai 30 e tio ficam pretos, avô 20 fica vermelho
  > Raiz 20 recolorida para preto
ID 50 inserido.
  > Inserção caso 3 (tio preto, linha): pai 40 fica preto, avô 30 fica vermelho, …
  >   rotação à esquerda em 30: 40 sobe
  …
Opção: 3
ID(s) a buscar: 25 99
ID 25 encontrado (filhos: 0, cor: vermelho) após 4 comparação(ões).
ID 99 não encontrado (3 comparação(ões)).
Opção: 2
ID(s) a remover: 10      ← folha preta
  > Remoção caso 1 (irmão 40 vermelho): recolorir e rotacionar no pai 20
  > Remoção caso 3 (sobrinho próximo 25 vermelho): recolorir e rotacionar no irmão 30
  > Remoção caso 4 (sobrinho distante 30 vermelho): recolorir e rotacionar no pai 20
ID(s) a remover: 40      ← dois filhos
  > Nó 40 tem dois filhos: será substituído pelo sucessor 50
  > Remoção caso 1 (irmão 25 vermelho): recolorir e rotacionar no pai 50
  > Remoção caso 2 (irmão 30 preto, sobrinhos pretos): irmão fica vermelho, sobe para 50
  > Nó 50 (vermelho) recolorido para preto, repondo o preto removido do caminho
```

visualização após as inserções

```
        /-- 50 [P]
    /-- 40 [V]
    |   \-- 30 [P]
    |       \-- 25 [V]
    20 [P]
    \-- 10 [P]

[P] = preto, [V] = vermelho
Altura negra = 2
```

estatísticas ao final

```
Opção: 6
Altura ..... 2
Rotações ..... 6
Recolorações ..... 23
Propriedades ..... OK
```

07

Testes: 37 de 37 aprovados

Cenário (mesmos dados nas duas árvores)	AVL	Rubro-Negra
Inserção crescente, decrescente e aleatória (2.000 IDs)	OK	OK
Valores repetidos rejeitados sem alterar a estrutura	OK	OK
Busca de IDs existentes e inexistentes (inclusive extremos)	OK	OK
Remoção de folha, de nó com 1 filho e com 2 filhos (inclusive raiz)	OK	OK
Remoção de inexistente e do mesmo ID duas vezes	OK	OK
Árvore vazia e árvore com um único elemento	OK	OK
Esvaziar a árvore em 3 ordens e por grau dos nós	OK	OK
Rotação simples (1, 2, 3) e dupla (30, 10, 20) com contagem exata	OK	OK
Altura dentro do limite teórico com 100.000 IDs	OK	OK
30.000 operações aleatórias comparadas com java.util.TreeSet	OK	OK

37/37

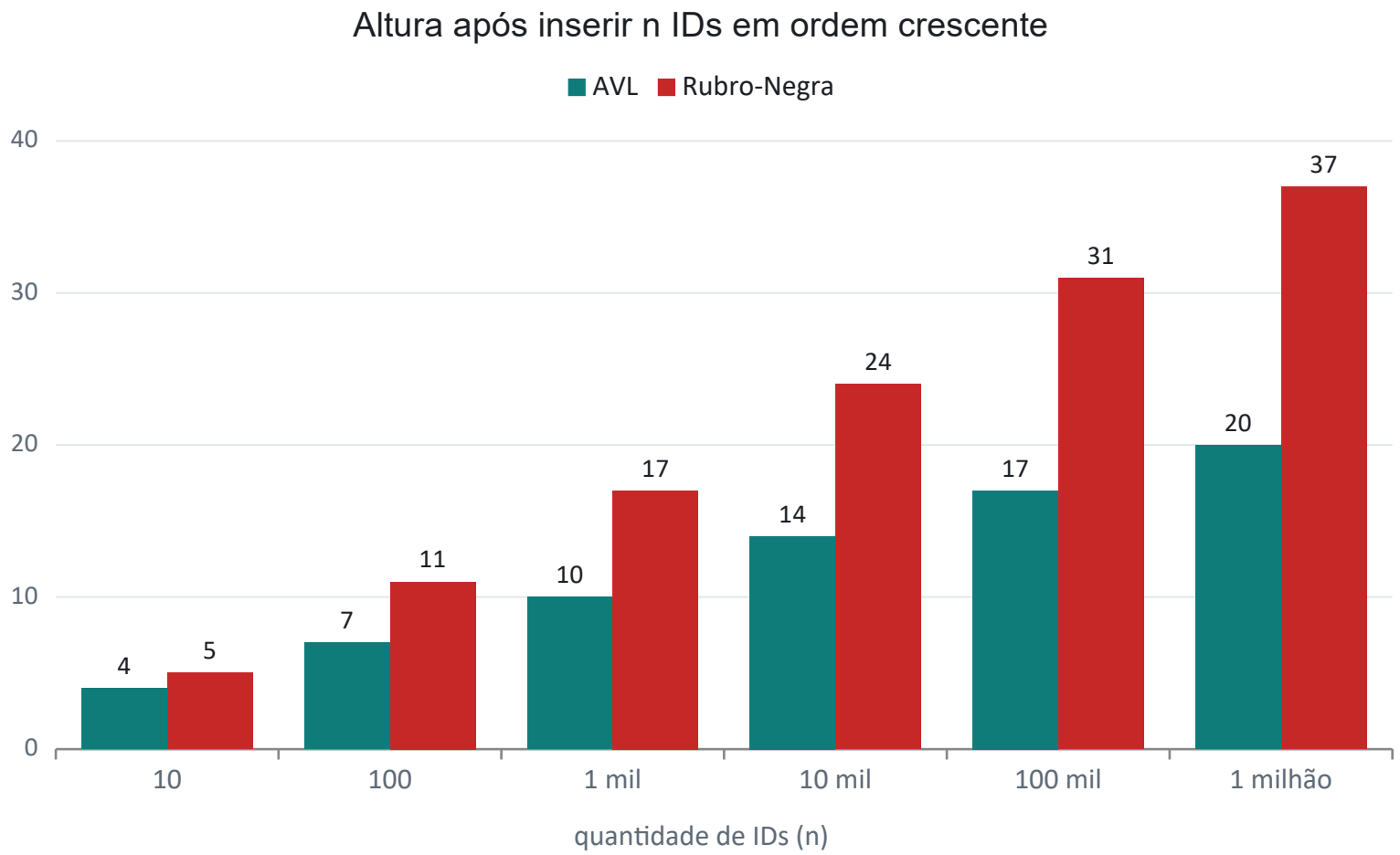
testes aprovados (18 cenários × 2 árvores + cores da Rubro-Negra)

Propriedades conferidas após cada inserção e remoção:
ordem de busca, alturas e $|FB| \leq 1$ (AVL); raiz preta, sem vermelho-vermelho e altura negra igual (Rubro-Negra)

```
java -jar … --testes
```

08

Análise: altura final com os mesmos dados



17 × 31

AVL x Rubro-Negra com 100 mil IDs sequenciais. A mínima possível é 17

20 × 20

com 100 mil IDs aleatórios: empate

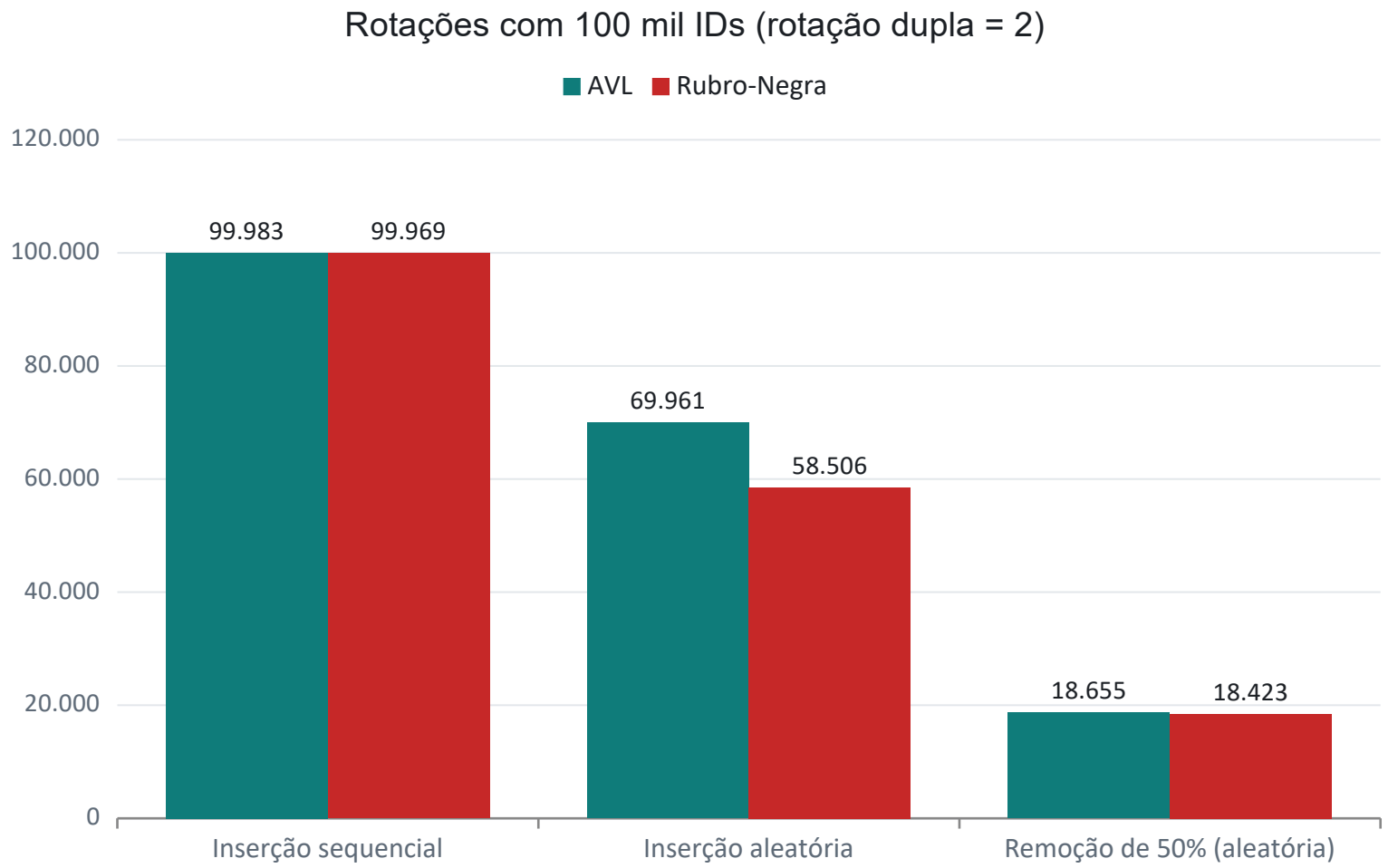
≈ 2 $\log_2$ n

a Rubro-Negra encosta no seu limite teórico com IDs sequenciais (31 de 33,2)

Ordem decrescente: resultados idênticos.

08

Análise: custo do balanceamento



≈ empate

com IDs sequenciais: cerca de 1 rotação por inserção nas duas

−16%

rotações da Rubro-Negra com IDs aleatórios

≈ 5

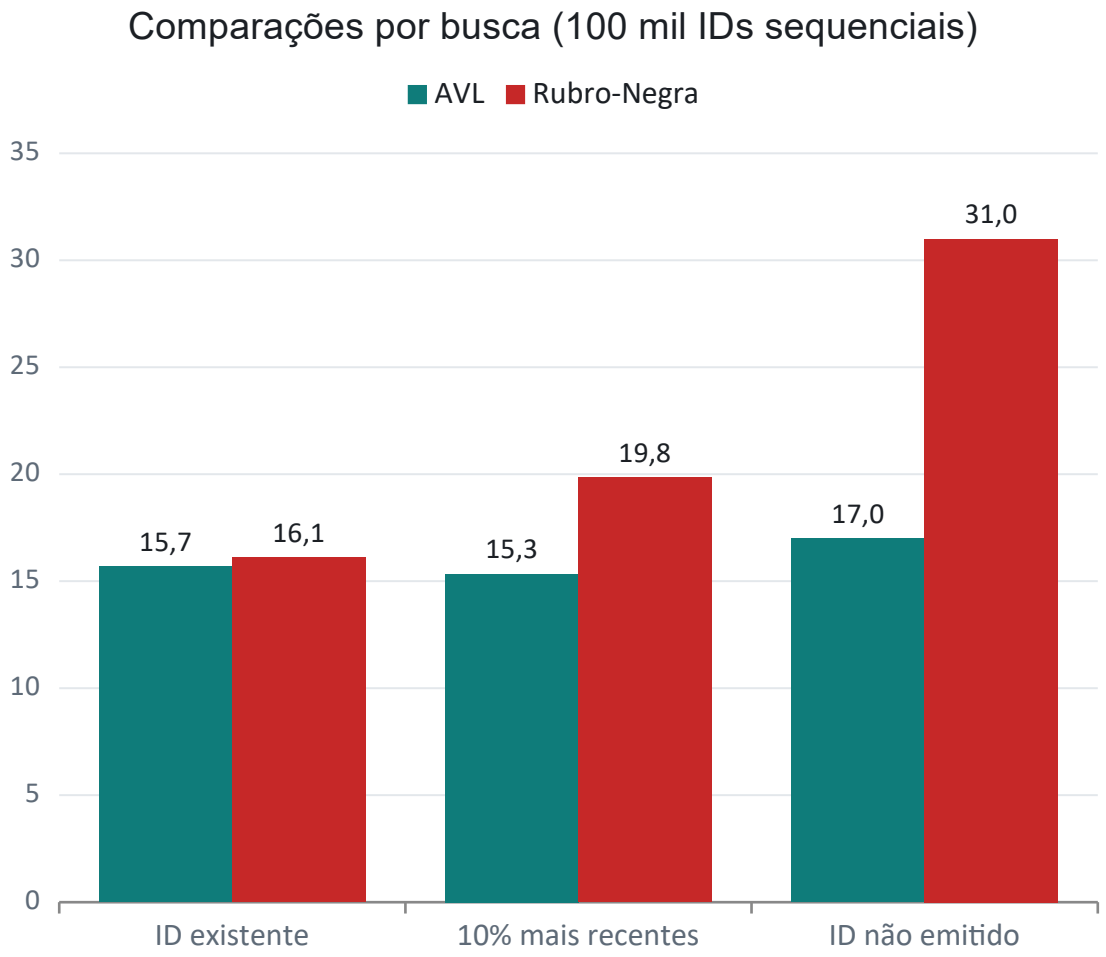
recolorações por inserção sequencial na Rubro-Negra (499.846 no total)

15–28%

inserção mais rápida na Rubro-Negra (mediana de 11 execuções)

08

Análise: custo da busca e cargas mistas



Carga (1 milhão de operações)	Altura AVL / RN	Comp./busca AVL / RN	Tempo (ms) AVL / RN
Rastreamento 10/85/5	17 / 24	13,9 / 14,1	109 / 138
Equilibrada 40/40/20	19 / 29	15,8 / 16,1	254 / 294
Pico de pedidos 45/10/45	19 / 29	15,6 / 16,0	289 / 256

Percentuais = inserção / busca / remoção, com IDs sequenciais. Os tempos variaram até 20% entre execuções e chegaram a inverter; altura e comparações são determinísticas.

–23%

comparações na AVL ao rastrear as encomendas mais recentes (15,3 vs 19,8): as recentes ficam no ramo mais fundo da Rubro-Negra

08 Diferenças, vantagens e limitações

Árvore AVL

Estratégia
balanceamento rígido por altura (|FB| ≤ 1)

Vantagens
árvore mais baixa (igual à mínima com IDs sequenciais); buscas mais curtas e previsíveis; nó simples

Limitações
atualiza alturas no caminho todo; remoção pode rotacionar em vários níveis; inserção 17–40% mais lenta

Aplicações
índices em memória com leitura predominante: consulta, rastreamento, dicionários

Árvore Rubro-Negra

Estratégia
balanceamento frouxo por cores ($h \leq 2 \log_2(n + 1)$)

Vantagens
≤ 2 rotações por inserção e ≤ 3 por remoção; 16% menos rotações com chaves aleatórias; inserção mais rápida

Limitações
até 82% mais alta com chaves sequenciais; mais casos no código; ~5 recolorações por inserção

Aplicações
bibliotecas de uso geral (java.util.TreeMap, std::map), escalonador CFS do Linux, cargas de escrita

09

Recomendação: Árvore AVL

Para o índice de encomendas da Black Friday: IDs chegam em sequência e o rastreamento é a operação dominante.

45%

menor altura com IDs sequenciais (17 vs 31 com 100 mil IDs)

23%

menos comparações ao rastrear as encomendas recentes (15,3 vs 19,8)

≈ 0

diferença em rotações nesse cenário (99.983 vs 99.969)

Quando a Rubro-Negra seria a escolha: chaves em ordem aleatória com escrita predominante, ou quando é preciso limitar a reestruturação por operação (estruturas concorrentes ou persistentes). Com chaves aleatórias, ela fez 16% menos rotações e inseriu cerca de 15% mais rápido nos testes.

10 Referências e repositório

Repositório público

github.com/LeoRosendof/Sistema-de-Arvores

- Código-fonte das duas árvores
- Versão compilada: dist/arvores-balanceadas.jar
- README com execução, dependências e exemplos
- Registros de execução e resultados do benchmark

Requisito: Java 17 ou superior

Referências

CORMEN, T. H. et al. Algoritmos: Teoria e Prática. 3. ed. Elsevier, 2012. Cap. 12 e 13.

ADELSON-VELSKY, G. M.; LANDIS, E. M. An algorithm for the organization of information. Soviet Mathematics Doklady, v. 3, 1962.

GUIBAS, L. J.; SEDGEWICK, R. A dichromatic framework for balanced trees. 19th FOCS, IEEE, 1978.

KNUTH, D. E. The Art of Computer Programming, v. 3: Sorting and Searching. 2. ed. Addison-Wesley, 1998.

SEDGEWICK, R.; WAYNE, K. Algorithms. 4. ed. Addison-Wesley, 2011. Seção 3.3.

SZWARCFITER, J. L.; MARKENZON, L. Estruturas de Dados e seus Algoritmos. 3. ed. LTC, 2010.

Uso de inteligência artificial

Desenvolvido com apoio do Claude (Anthropic), assistente de IA usado na revisão do código e dos testes, na conferência dos resultados e em correções desta apresentação. Decisões, análise e recomendação são do autor; detalhes na seção 9 do README.